

1.(1)~(4)

```
In [1]: def digit_count(n):
          count = 0
          while n > 0:
              n //= 10
              count += 1
          return count
          digit_count(8888888888888888)
```

Out[1]: 15

```
In [2]: def hundred_rec(n):  
         n=n // 100  
         n = n % 10  
         return n  
         hundred_rec(87)
```

Out[2]: 0

```
In [3]: A=[
    [-1111,2,3],
    [4,5,6]
]
def matrix_min(n):
    vec=[]
    for i in range(len(n)):
        for j in range(len(n[i])):
            vec.append(n[i][j])
    min_val=min(vec)
    return min_val
matrix_min(A)
```

```
Out[3]: -1111
```

```
In [4]: #判断方差就好，如果方差是0，肯定都相等

def ismatch(n):
    val_sum=0
    for i in range(len(n)):
        val_sum=val_sum + n[i-1]
    ave=val_sum/len(n)

    fangcah=0
    for i in range(len(n)):
        squ=(n[i]-ave)*(n[i]-ave)
        fangcha=fangcah+squ
    if fangcha == 0:
        return "same"
    else:
        return "diff"

B=[1,0,9,1]
C=[0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
```

```
print(ismatch(B))
print(ismatch(C))
```

diff
same

2.

```
In [7]: def find_inverse_permutation(a):
        """
        计算置换 a 的逆置换。

        参数:
        a: 一个列表, 包含 1 到 n 的整数, 是 1..n 的一个重排。

        返回:
        一个列表, 表示 a 的逆置换。
        """
        n = len(a)

        inv_a = [0] * n

        for i in range(n):
            # 原映射: 位置 (i+1) -> 值 a[i]
            # 逆映射: 值 a[i] -> 位置 (i+1)

            # 在逆置换列表中, 值 a[i] 对应的索引是 a[i]-1
            # 该位置应填入原始位置 i+1
            value_from_a = a[i]
            original_position = i + 1

            inv_a[value_from_a - 1] = original_position

        return inv_a

test_a2 = [1, 2, 3, 4]
result2 = find_inverse_permutation(test_a2)
print(f"原置换: {test_a2}")
print(f"逆置换: {result2}")
```

原置换: [1, 2, 3, 4]
逆置换: [1, 2, 3, 4]

3.

```
In [1]: import math

def calculate_pi_approximation(terms):

    # 1. 计算 x = 1 / sqrt(3)
    # 也就是 sqrt(3)/3
    x = 1.0 / math.sqrt(3.0)

    # 2. 初始化 arctan(x) 的和
    atan_sum = 0.0
```

```

# 3. 累加泰勒级数的前 terms 项
# 通项公式:  $(-1)^n * x^{(2n+1)} / (2n+1)$ , 其中 n 从 0 开始
for n in range(terms):
    # 计算指数
    power = 2 * n + 1

    # 计算当前项的值:  $x^{power} / power$ 
    term_value = (x ** power) / power

    # 根据  $(-1)^n$  决定符号
    if n % 2 == 0:
        atan_sum += term_value
    else:
        atan_sum -= term_value

# 4. 计算  $\pi = 6 * \arctan(x)$ 
pi_approx = 6 * atan_sum

return pi_approx

real_pi = math.pi

print(f"{'项数':<10} {'近似值':<20} {'绝对误差':<20}")
print("-" * 50)

# 测试不同的项数, 观察收敛情况
test_terms = [1, 2, 5, 10, 20, 50, 100]

for t in test_terms:
    approx_pi = calculate_pi_approximation(t)
    error = abs(approx_pi - real_pi)
    print(f"{'t':<10} {'approx_pi':<20.15f} {'error':<20.15e}")

```

项数	近似值	绝对误差
1	3.464101615137755	3.225089615479622e-01
2	3.079201435678004	6.239121791178892e-02
5	3.142604745663085	1.012092073291448e-03
10	3.141590510938080	2.142651712944144e-06
20	3.141592653571403	1.839017826910094e-11
50	3.141592653589794	4.440892098500626e-16
100	3.141592653589794	4.440892098500626e-16

4.

In [9]: `def calculate_euler_constant(digits=10):`

```

    n = 1000000

    harmonic_sum = 0.0
    for k in range(1, n + 1):
        harmonic_sum += 1.0 / k
    ln_n = math.log(n)
    term_1 = 1.0 / (2 * n)
    term_2 = 1.0 / (12 * n * n)

```

```

gamma_approx = harmonic_sum - ln_n - term_1 + term_2

return gamma_approx

if __name__ == "__main__":
    # 计算近似值
    gamma_val = calculate_euler_constant()

    # 真实值用于对比 (来自数学常数表)
    # gamma ≈ 0.577215664901532...
    real_gamma = 0.57721566490153286060651209

    print(f"计算使用的 n = 1,000,000")
    print(f"修正项: -1/(2n) + 1/(12n^2)")
    print("-" * 40)
    print(f"计算得到的 γ: {gamma_val:.15f}")
    print(f"真实的 γ:      {real_gamma:.15f}")
    print(f"绝对误差:      {abs(gamma_val - real_gamma):.2e}")

    # 格式化输出精确到 10 位小数
    print("-" * 40)
    print(f"精确到 10 位小数的结果: {gamma_val:.10f}")

```

计算使用的 n = 1,000,000

修正项: $-1/(2n) + 1/(12n^2)$

计算得到的 γ: 0.577215664900799

真实的 γ: 0.577215664901533

绝对误差: 7.34e-13

精确到 10 位小数的结果: 0.5772156649

5.

```

In [4]: import numpy as np
import matplotlib.pyplot as plt

def f(x):
    """
    定义分段函数 f(x)
    """
    # 初始化结果数组, 与 x 形状相同, 初始值为 0 (对应 |x| >= 2 的情况)
    y = np.zeros_like(x, dtype=float)

    # 条件 1: |x| <= 1 => -1 <= x <= 1
    mask1 = (x >= -1) & (x <= 1)
    y[mask1] = 2 - x[mask1]**2

    # 条件 2: 1 < x < 2
    mask2 = (x > 1) & (x < 2)
    y[mask2] = (x[mask2] - 2)**2

    # 条件 3: -2 < x < -1
    mask3 = (x > -2) & (x < -1)
    y[mask3] = (x[mask3] + 2)**2

```

```
# 条件 4:  $|x| \geq 2$  已经在初始化时设为 0, 无需额外操作
# 但为了严谨, 可以显式赋值 (可选)
# mask4 = (x <= -2) | (x >= 2)
# y[mask4] = 0

return y

# 生成 x 数据点
# 范围稍微超出 [-2, 2] 以显示两侧的 0
x = np.linspace(-3, 3, 1000)
y = f(x)

# 创建图像
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='$f(x)$', color='blue', linewidth=2)

# 添加关键点的标记, 以便观察分段连接处
critical_points_x = [-2, -1, 1, 2]
critical_points_y = [f(np.array([pt]))[0] for pt in critical_points_x]
plt.scatter(critical_points_x, critical_points_y, color='red', zorder=5, label='Key')

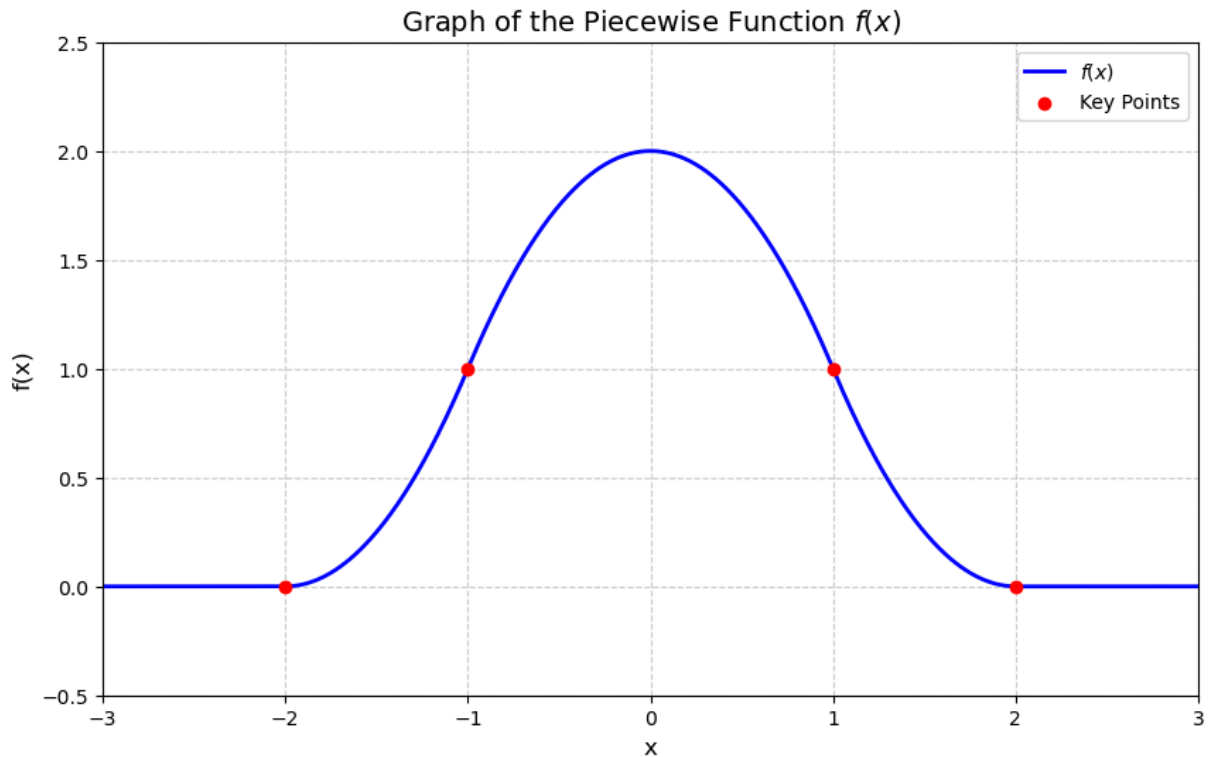
# 设置标题和标签
plt.title('Graph of the Piecewise Function $f(x)$', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('f(x)', fontsize=12)

# 添加网格
plt.grid(True, linestyle='--', alpha=0.6)

# 添加图例
plt.legend()

# 设置坐标轴范围, 使图像更美观
plt.ylim(-0.5, 2.5)
plt.xlim(-3, 3)

# 显示图像
plt.show()
```



6.

```
In [ ]: def reorder_symmetric_matrix(A):

    # 转换为 numpy 数组以便操作
    A = np.array(A, dtype=float)
    n = A.shape[0]

    # 1. 提取对角线元素
    diag_elements = np.diag(A)

    # 2. 计算对角线元素的绝对值
    abs_diag = np.abs(diag_elements)

    # 3. 获取按绝对值从大到小排序的索引
    # argsort 默认是从小到大, 所以取负号或者反转
    sorted_indices = np.argsort(-abs_diag)

    # 4. 根据排序后的索引重排矩阵
    # B[i, j] = A[sorted_indices[i], sorted_indices[j]]
    # 这等于先换行再换列, 或者直接使用高级索引
    B = A[np.ix_(sorted_indices, sorted_indices)]

    return B

# --- 测试代码 ---
if __name__ == "__main__":
    # 示例 1: 一个简单的 3x3 对称矩阵
    # 对角线: 1, 5, 2 -> 绝对值: 1, 5, 2
    # 期望顺序: 索引 1 (值5), 索引 2 (值2), 索引 0 (值1)
    A1 = [
```

```
[1, 0.5, 0.2],
[0.5, 5, 0.8],
[0.2, 0.8, 2]
]

print("原始矩阵 A:")
print(np.array(A1))
print("原始对角线:", np.diag(A1))

B1 = reorder_symmetric_matrix(A1)
print("\n重排后的矩阵 B:")
print(B1)
print("重排后对角线:", np.diag(B1))
print("对角线绝对值:", np.abs(np.diag(B1)))

# 验证对称性
print("是否保持对称?", np.allclose(B1, B1.T))

print("-" * 30)

# 示例 2: 包含负数的对角线
# 对角线: -10, 3, -1 -> 绝对值: 10, 3, 1
# 期望顺序: 索引 0 (-10), 索引 1 (3), 索引 2 (-1)
A2 = [
    [-10, 1, 2],
    [1, 3, 4],
    [2, 4, -1]
]

print("原始矩阵 A2:")
print(np.array(A2))

B2 = reorder_symmetric_matrix(A2)
print("\n重排后的矩阵 B2:")
print(B2)
print("重排后对角线:", np.diag(B2))

# 验证对称性
print("是否保持对称?", np.allclose(B2, B2.T))
```

原始矩阵 A:

```
[[1.  0.5 0.2]
 [0.5 5.  0.8]
 [0.2 0.8 2.  ]]
```

原始对角线: [1. 5. 2.]

重排后的矩阵 B:

```
[[5.  0.8 0.5]
 [0.8 2.  0.2]
 [0.5 0.2 1.  ]]
```

重排后对角线: [5. 2. 1.]

对角线绝对值: [5. 2. 1.]

是否保持对称? True

原始矩阵 A2:

```
[[-10  1  2]
 [ 1  3  4]
 [ 2  4 -1]]
```

重排后的矩阵 B2:

```
[[-10.  1.  2.]
 [ 1.  3.  4.]
 [ 2.  4. -1.]]
```

重排后对角线: [-10. 3. -1.]

是否保持对称? True

7.

```
In [7]: def horner_method(coeffs, x):
        """
        使用霍纳方法计算多项式的值。

        参数:
        coeffs: 列表, 包含多项式系数 [a_n, a_{n-1}, ..., a_1, a_0]
                即第一个元素是最高次项系数, 最后一个元素是常数项。
        x: 变量 x 的值

        返回:
        p(x) 的计算结果
        """
        if not coeffs:
            return 0

        # 初始化 u_n = a_n
        # coeffs[0] 对应 a_n
        u = coeffs[0]

        # 从 k = n-1 到 0 进行迭代
        # 在列表中, 这对应于 coeffs[1] 到 coeffs[-1]
        for i in range(1, len(coeffs)):
            # a_k 对应 coeffs[i]
            # u_{k} = u_{k+1} * x + a_k
            u = u * x + coeffs[i]

        return u
```

```
# --- 测试 ---
if __name__ == "__main__":
    # 示例:  $p(x) = 2x^3 - 6x^2 + 2x - 1$ 
    # 系数:  $a_3=2, a_2=-6, a_1=2, a_0=-1$ 
    # coeffs = [2, -6, 2, -1]

    coefficients = [2, -6, 2, -1]
    x_val = 3

    result = horner_method(coefficients, x_val)

    # 手动验证:
    #  $p(3) = 2*(3^3) - 6*(3^2) + 2*(3) - 1$ 
    #      =  $2*27 - 6*9 + 6 - 1$ 
    #      =  $54 - 54 + 6 - 1$ 
    #      = 5

    print(f"多项式系数 (高->低): {coefficients}")
    print(f"x = {x_val}")
    print(f"霍纳方法计算结果: {result}")
    print(f"预期结果: 5")
```

多项式系数 (高->低): [2, -6, 2, -1]

x = 3

霍纳方法计算结果: 5

预期结果: 5

```
In [9]: # 1. 生成 x 数据点
# 范围设为 -5 到 5, 包含 400 个点以保证曲线平滑
x = np.linspace(-5, 5, 400)

# 2. 计算各个函数的 y 值
y1 = np.ones_like(x)          #  $f_1(x) = 1$ 
y2 = 1 / (x**2 + 1)           #  $f_2(x) = 1 / (x^2 + 1)$ 
y3 = np.sin(x) / (np.exp(x) + 1) #  $f_3(x) = \sin(x) / (e^x + 1)$ 

# 3. 创建图像窗口
plt.figure(figsize=(10, 6))

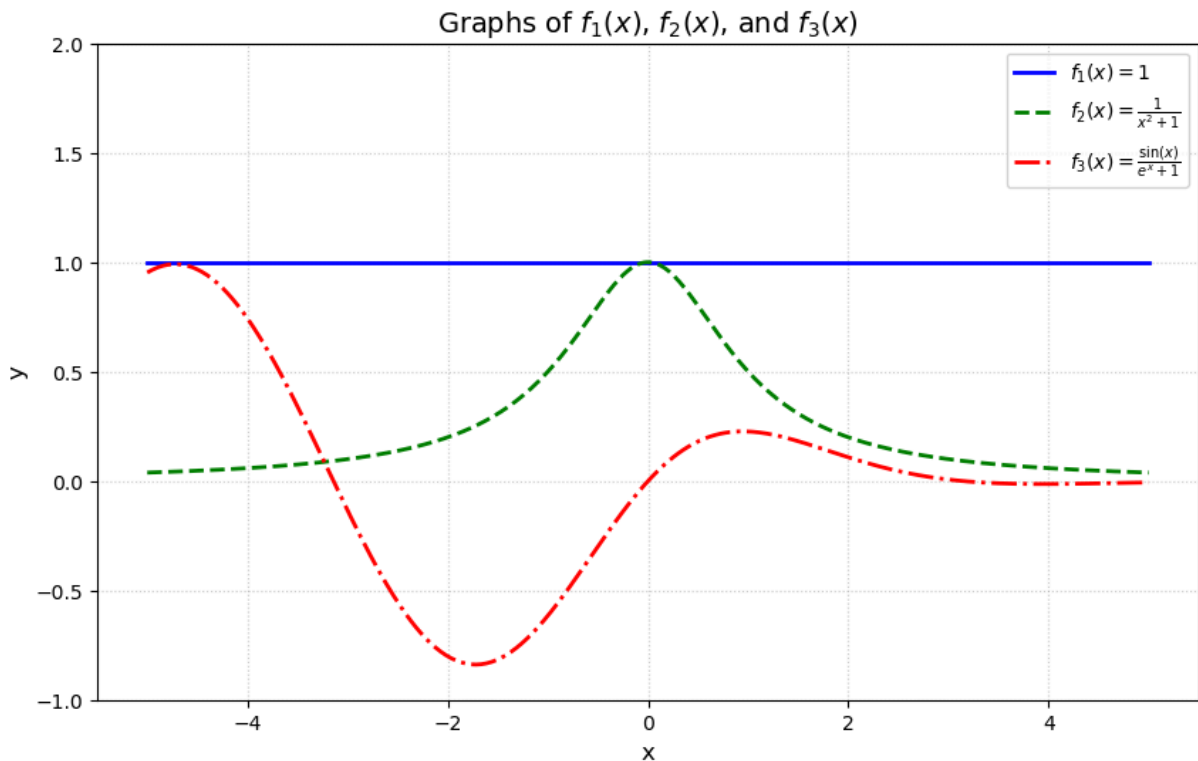
# 4. 绘制三条曲线
plt.plot(x, y1, label='$f_1(x) = 1$', color='blue', linestyle='-', linewidth=2)
plt.plot(x, y2, label='$f_2(x) = \frac{1}{x^2+1}$', color='green', linestyle='--',
plt.plot(x, y3, label='$f_3(x) = \frac{\sin(x)}{e^x+1}$', color='red', linestyle='

# 5. 添加标题、标签和图例
plt.title('Graphs of  $f_1(x)$ ,  $f_2(x)$ , and  $f_3(x)$ ', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend(loc='best')

# 6. 添加网格线
plt.grid(True, linestyle=':', alpha=0.6)

# 7. 设置坐标轴范围 (可选, 为了让图像更美观)
plt.ylim(-1, 2)
```

```
# 8. 显示图像
plt.show()
```



```
In [11]: from mpl_toolkits.mplot3d import Axes3D

def plot_hyperboloid(a, b, c):
    """
    绘制单叶双曲面  $x^2/a^2 + y^2/b^2 - z^2/c^2 = 1$ 
    """
    # 1. 创建参数网格
    # theta: 0 到 2*pi
    # u: -2 到 2 (可以根据需要调整范围以改变高度)
    theta = np.linspace(0, 2 * np.pi, 50)
    u = np.linspace(-2, 2, 50)

    # 生成网格矩阵
    theta, u = np.meshgrid(theta, u)

    # 2. 计算坐标点
    x = a * np.cos(theta) * np.cosh(u)
    y = b * np.sin(theta) * np.cosh(u)
    z = c * np.sinh(u)

    # 3. 创建 3D 图形
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    # 4. 绘制曲面
    # cmap='viridis' 设置颜色映射, alpha=0.8 设置透明度
    ax.plot_surface(x, y, z, cmap='viridis', edgecolor='none', alpha=0.8)
```

```

# 5. 设置标题和标签
title = f'Single-sheeted Hyperboloid\n$a={a}, b={b}, c={c}$'
ax.set_title(title, fontsize=14)
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Z axis')

# 6. 设置坐标轴比例一致 (可选, 防止变形)
# 注意: 在较新版本的 matplotlib 中 set_box_aspect 可能更适用, 或者手动设置 limits
max_range = np.array([x.max()-x.min(), y.max()-y.min(), z.max()-z.min()]).max()
mid_x = (x.max()+x.min()) * 0.5
mid_y = (y.max()+y.min()) * 0.5
mid_z = (z.max()+z.min()) * 0.5
ax.set_xlim(mid_x - max_range, mid_x + max_range)
ax.set_ylim(mid_y - max_range, mid_y + max_range)
ax.set_zlim(mid_z - max_range, mid_z + max_range)

plt.show()

if __name__ == "__main__":
    try:
        # 获取用户输入
        print("请输入单叶双曲面参数 a, b, c 的值:")
        a_val = float(input("a = "))
        b_val = float(input("b = "))
        c_val = float(input("c = "))

        if a_val == 0 or b_val == 0 or c_val == 0:
            print("错误: a, b, c 不能为 0")
        else:
            plot_hyperboloid(a_val, b_val, c_val)

    except ValueError:
        print("输入无效, 请输入数字。")
    except Exception as e:
        print(f"发生错误: {e}")

```

请输入单叶双曲面参数 a, b, c 的值:

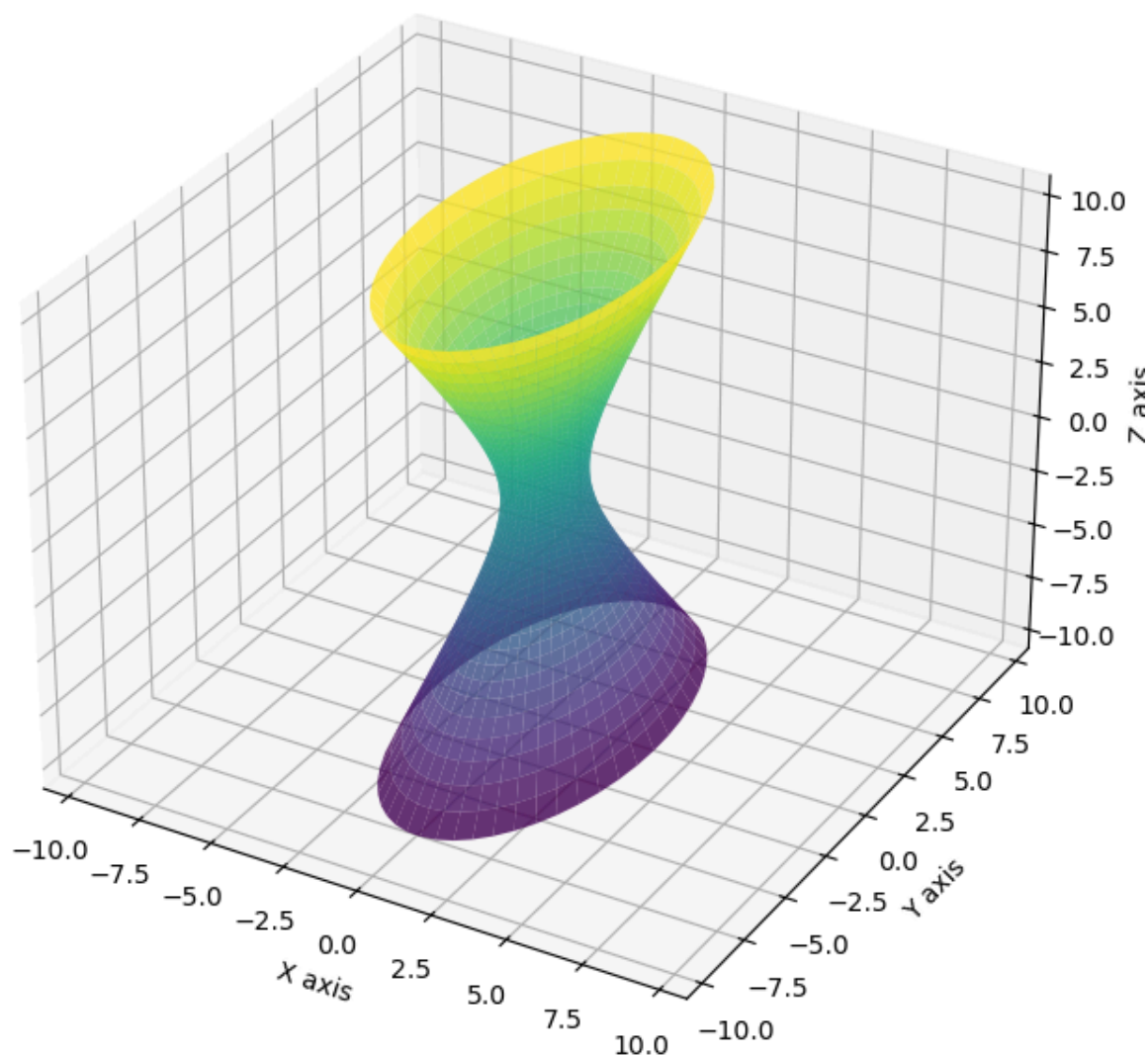
a = 1

b = 2

c = 3

Single-sheeted Hyperboloid

$$a = 1.0, b = 2.0, c = 3.0$$



```
In [12]: def euclidean_norm_safe(x):
    """
    计算向量 x 的欧几里得范数，采用缩放法避免上溢和下溢。

    参数:
    x: 列表或元组，包含数值

    返回:
    欧几里得范数 (float)
    """
    if not x:
        return 0.0

    n = len(x)

    # 1. 找到绝对值最大的元素 m
    # 使用 abs() 获取绝对值, max() 获取最大值
```

```

m = 0.0
for val in x:
    abs_val = abs(val)
    if abs_val > m:
        m = abs_val

# 2. 如果最大值为 0, 则向量为零向量, 范数为 0
if m == 0.0:
    return 0.0

# 3. 计算缩放后的平方和
#  $\sum (x_i / m)^2$ 
scaled_sum_sq = 0.0
for val in x:
    # 先除法再平方, 确保中间值在 [-1, 1] 之间
    scaled_val = val / m
    scaled_sum_sq += scaled_val * scaled_val

# 4. 恢复尺度:  $\|x\| = m * \sqrt{\sum (x_i/m)^2}$ 
norm = m * math.sqrt(scaled_sum_sq)

return norm

# --- 测试代码 ---
if __name__ == "__main__":
    # 测试 1: 普通向量
    v1 = [3, 4]
    print(f"向量 {v1} 的范数: {euclidean_norm_safe(v1)}") # 应为 5.0

    # 测试 2: 可能导致上溢的大数
    # 1e200 的平方是 1e400, 超出 double 范围
    v2 = [1e200, 1e200]
    print(f"大数向量范数: {euclidean_norm_safe(v2)}")
    # 直接计算  $\sqrt{1e400 + 1e400}$  会得到 inf
    # 安全计算:  $m=1e200, \text{sum\_sq} = 1^2 + 1^2 = 2, \text{result} = 1e200 * \sqrt{2} \approx 1.414e200$ 

    # 测试 3: 可能导致下溢的小数
    # 1e-200 的平方是 1e-400, 低于 double 最小精度, 变为 0
    v3 = [1e-200, 1e-200]
    print(f"小数向量范数: {euclidean_norm_safe(v3)}")
    # 直接计算  $\sqrt{0 + 0}$  会得到 0.0 (错误)
    # 安全计算:  $m=1e-200, \text{sum\_sq} = 1^2 + 1^2 = 2, \text{result} = 1e-200 * \sqrt{2} \approx 1.414e-200$ 

    # 对比: 直接使用 naive 方法的问题演示
    def naive_norm(x):
        s = 0
        for val in x:
            s += val * val
        return math.sqrt(s)

    print("-" * 30)
    print("Naive 方法处理大数:", naive_norm(v2)) # 输出 inf
    print("Naive 方法处理小数:", naive_norm(v3)) # 输出 0.0

```

```
向量 [3, 4] 的范数: 5.0  
大数向量范数: 1.414213562373095e+200  
小数向量范数: 1.414213562373095e-200  
-----  
Naive 方法处理大数: inf  
Naive 方法处理小数: 0.0
```